Reflection Report

Teleoperated medical transport robot for hospital environment

Name: Adhithya Palanisamy Chandrasekaran

Course: Telerobotics and Applied Sensor Fusion (7.5 ECTS)

January 25, 2026

Contents

1 Project context and area of focus **2**

2 Activities and estimated hours **2**

2.1 Teleoperation + ROS2 networking 2

2.2 Return path generation + smoothing and planning experiments 2

2.3 Collision avoidance integration during return 3

2.4 State estimation experiments: heading Kalman filter 3

2.5 Multi-goal routing (TSP) + "robot solving a real problem" modeling 3

3 Fulfilment of Expected Tasks **3**

4 Group work and collaboration **4**

5 Manual teleportation **4**

6 Path generation **5**

7 Collision avoidance **7**

8 Kalman Filter **7**

9 TSP and Robot solving a real problem **8**

9.1 Hospital Logistics Delivery Replenishment Problem (H-LDRP) 8

10 Gazebo Simulation attempt **9**

11 Development experience and challenges **10**

12 Reflections and Proposed Improvements **10**

13 Societal and ethical reflections **11**

14 Conclusion **11**

1 Project context and area of focus

Our project aimed to build a remotely-operated differential-drive robot suitable for a hospital-like environment (e.g., delivering medicines, small equipment, or samples), where the operator can teleoperate the vehicle and the robot can also return safely to the starting position with obstacle handling. This matches the project directive's emphasis on a Teleoperator, Estimator (Kalman Filter), Controller, and optionally a Simulator, all communicating over ROS2.

My contributions focused on the teleoperation workflow, return-path generation and improvement ideas (sampling + smoothing/planning), collision avoidance integration, and state-estimation experiments (Kalman heading fusion). I also explored "robot does work" problems beyond point-to-point motion: multi-stop routing (TSP) and a hospital-adapted version of a logistics allocation/scheduling problem inspired by part-feeding in industry.

2 Activities and estimated hours

Below is an approximate time log (excluding lectures). I'm giving realistic ranges because my work happened in bursts around lab access, debugging sessions, and integration days.

2.1 Teleoperation + ROS2 networking

- Implemented and tested the operator-side joystick teleoperation node using ***pygame***, publishing ***cmd_vel*** and a ***trigger_return*** toggle.
- Added safe behavior: deadzone, high-rate publish loop, and a toggle that stops sending ***cmd_vel*** when return mode is activated (to avoid command conflicts).
- Verified ROS2 discovery and connectivity over Wi-Fi, and helped the team standardize how we launched teleop and return.

2.2 Return path generation + smoothing and planning experiments

- Implemented and tuned the **"record while manual driving → return by replay"** idea.
- Experimented with multiple variants:
  1. **Simple sampling-based return path**: reverse the recorded pose list and subsample waypoints (every ***N*** samples), more like a **teach-and-repeat / breadcrumb trail navigation**.
  2. **Gaussian smoothing** of the recorded path (applying gaussian_filter1d independently to x and y) to reduce jitter and produce a more planned looking path for plotting and for smoother control.
  3. **Grid-based Planning (A*)** using the map, then post-processing using:
     - **String-pulling / shortcutting** - remove intermediate points if line-of-sight is free via Bresenham traversal, and
     - **Moving-average** smoothing on final waypoints.

2.3 Collision avoidance integration during return

- Integrated LiDAR scan processing into a finite-state avoidance behavior, only active during return.
- Focus: avoiding oscillations and improving the path rejoining behavior (hysteresis, cooldown/commit windows, scanning for visible future waypoints, etc.).
- Validated in hallway tests that the robot can detour around obstacles and continue returning.

2.4 State estimation experiments: heading Kalman filter

- Implemented heading Kalman filter variants that fuse **gyro-based** prediction with heading observations from **wheel encoders** and optionally a heading estimate from IMU integration.
- The prediction step integrates yaw rate (*gyro_z*) and the observation step corrects with encoder heading (*/robot/pose_enc*) and optionally IMU heading (*/robot/pose*).

2.5 Multi-goal routing (TSP) + "robot solving a real problem" modeling

- Tested a TSP solver concept for sequencing multiple delivery points (e.g., multiple wards/rooms). This is directly relevant in hospitals where route order matters for time, battery, and urgency.
- Adapted ideas from an assembly line feeding / logistics scheduling case study to a hospital setting.

In addition to these, I have also implemented a perfectly working frontier exploration algorithm to explore an unknown environment (it collides with another subject but it was first implemented in this robot, to see how accurate its mapping would be considering that it was built from scratch).

Yes, I completed what I was expected to do, and in several areas (A*/smoothing, TSP, allocation modeling, KF variants) I did more than the baseline, even if not all of it ended up fully integrated into the final demo.

3 Fulfilment of Expected Tasks

What I fully completed

- A working manual teleoperation node used by the group.
- Return mode conceptually consistent with the directive: operator drives out, robot returns automatically, with safety logic.
- Practical collision avoidance behavior during return.
- A* + shortcutting + smoothing path generation

What I partially completed (good progress, not final-integrated)

- Gazebo simulation in hospital world: attempted, but not finalized.

4 Group work and collaboration

We worked as a system-of-systems team, aligning well with the project directive's separation into units. In practice we mixed roles depending on lab access, I often bridged operator-side control + path/return logic + estimation experiments with also help to be done with SLAM mapping.

Meetings and organization

- We used short stand-ups during lab days (what works / what fails / what to test next).
- We also did troubleshooting "micro-meetings" when integration broke. we also used whats-app for communication.

Dependencies

- The group depended on my teleoperation node because manual driving was the starting requirement and was needed for mapping and for collecting return paths.

Collaboration

I collaborated most with the members who handled path planning and SLAM:

- **Andreas** : We worked different methods on path planning and return. We didn't work together on the same method but tried out different approaches parallely. As the manual teloperation node stores the path history as a JSON file it was used for return path planning, we did share ideas and working principles parallel and it kept me motivated.
- **Tim** : We collaborated on SLAM and especially figured out the optimal parameters and migration of the package to another system as the pi4 was not computationally efficient. Its the level of interest though coming from a different domain out of the engineering and bought realistic insights from what a medical perspective (his domain) should look like.

5 Manual teleportation

Operator side

The operator-side teleoperation node reads a physical game controller using pygame and publishes:

* *geometry_msgs/Twist on cmdz_vel* (linear.x forward/back, angular.z turning), and
* a *std_msgs/Bool* on */trigger_return* to switch the robot into return mode.

A deliberate safety choice was made: when return mode is triggered, the teleop node stops publishing *cmd_vel* (and sends a stop burst), preventing command conflicts between manual driving and autonomous return.

Robot side

On the robot, the manual teleoperation command (*cmd_vel*) is received by the robot-side control stack (motor driver/controller node), which converts velocity commands into differential wheel speeds for the MD25-driven EMG30 motors. In parallel, the robot continuously publishes state and sensor topics required by the rest of the system (pose/odometry, LiDAR scans, IMU). Our overall architecture follows ROS2 topic-based modularity (publish/subscribe), matching how the project report describes the system as interacting nodes rather than one monolithic program.

6 Path generation

The simplest return mode does not plan a new path in the map but instead it uses the fact that we already drove a safe route manually:

1. Record (x, y, theta) while teleoperating.
2. On return trigger, reverse the recorded list so the robot follows it back toward the start.
3. Sub-sample it (take every Nth pose) to create manageable waypoints.
4. Run a controller to drive waypoint-to-waypoint.
5. During return only, use LiDAR to temporarily enter avoidance states and later rejoin the path.



Figure 1: Manual teleoperation and automatic return path of Robot in SLAM.

This approach is surprisingly effective in structured corridors: it's like creating a "breadcrumb trail" that can be replayed.

Gaussian filter

- The core return controller can work purely with sampling and sub-selecting waypoints for return.
- The Gaussian filter (1D on x and y) is an additional improvement step experimented to smooth the recorded trajectory into a cleaner planned-looking curve, which is useful both for control smoothness. A sigma parameter determines more like a window determines the smoothness.

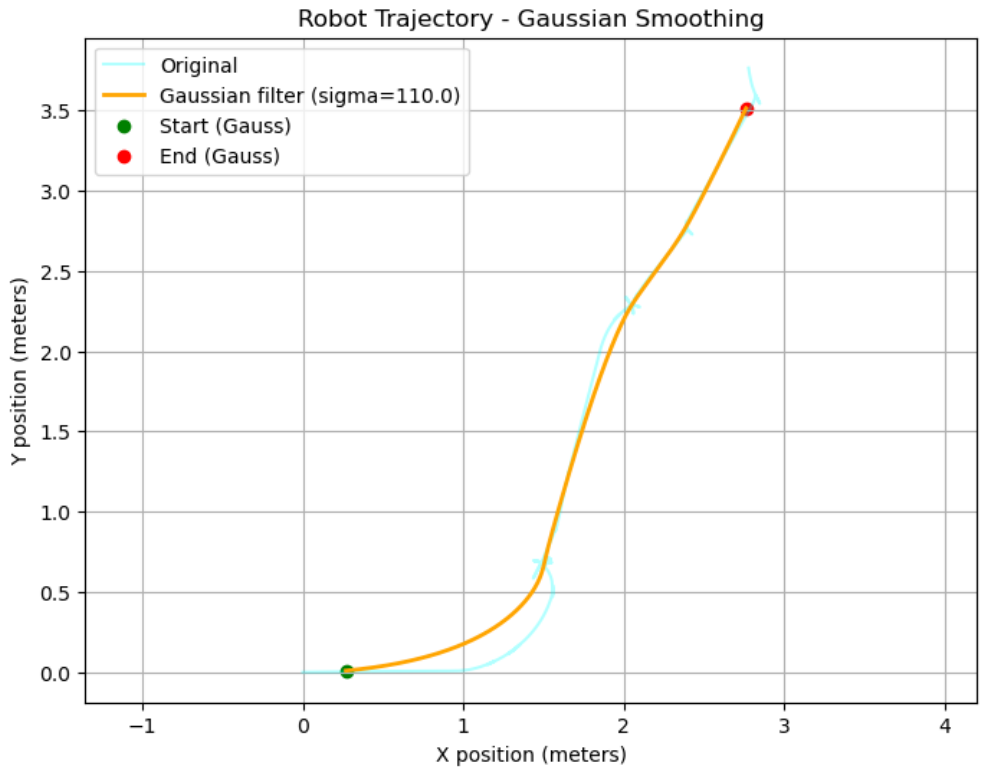


Figure 2: Gaussian filter applied to the pose history.

A* with string pulling + moving average

I also explored map-based planning (A*) primarily because hospitals may include:

- dynamic obstacles (people, carts), and
- cases where returning along the exact same path is not possible (blocked corridor).

A* generates a grid path, but raw A* has staircase artifacts, more like a jagged path. That's why shortcutting (string-pulling) and smoothing filters matter:

- **String-pulling** removes unnecessary intermediate nodes if straight-line visibility exists,
- **Moving average** smoothing reduces sharp turns and improves controller stability.

Even though I didn't fully deploy it in the final demo showed, it did work well with the frontier integration algorithm for this hospital robot, the exercise taught me that planning

is not just computing a path—it's producing a path that a real controller can track robustly, just because a path is short doesn't mean it is the best, when the path is smoothened then it allows the robot to travel at a higher velocity without sharp turns.

7 Collision avoidance

Collision avoidance looks simple at first ("if front is blocked, turn"), but the hard part is preventing oscillation and getting stuck:

- If you trigger avoidance too easily, the robot jitters and never commits forward progress.
- If you stay in avoidance too long, you may spiral away from the original path.

The most valuable insight I gained was the importance of state machines + hysteresis:

- AVOID_TURN until front is really clear (safety + hysteresis),
- AVOID_DRIVE to "escape" and search for a waypoint to rejoin,
- cooldown/commit windows to prevent immediate re-trigger loops.

This is the kind of practical autonomy that separates a demo from something dependable in a hospital corridor.

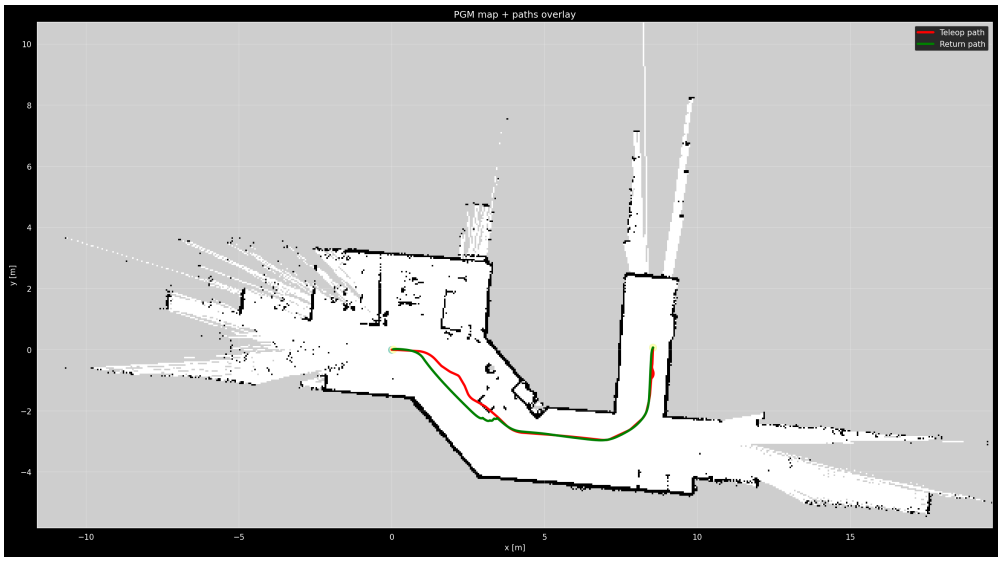


Figure 3: Robot collision avoidance during return.

8 Kalman Filter

In the heading fusion experiments, the key idea was:

Prediction (Process model)

Use IMU Yaw rate (gyro_z) integrated over time:

- **1D model**: $\theta_{k+1} = \theta_k + \dot{\theta}_{\text{gyro}}\,\Delta t$

  - **2-state model**: $\theta_{k+1} = \theta_k + (\dot{\theta}_{\text{gyro}} - b_k)\,\Delta t, \quad b_{k+1} = b_k$ where b is gyro bias

- **Observation (measurement update)** Correct the predicted heading using:

  encoder-derived heading (*/robot/pose_ enc*) as a measurement of θ.

  This reinforced a core sensor fusion lesson, that sensors aren't just noisy, they can be correlated and that matters for filter tuning.

9 TSP and Robot solving a real problem

In a hospital environment, a delivery robot rarely goes to one destination. A realistic mission would be:

- collect medicines from pharmacy/storage,
- deliver to multiple wards/rooms,
- optionally pick up samples/equipment,
- return to base to charge

The Traveling salesman problem (TSP) provides a principal way to decide the visiting order to reduce total cost which would be either time, distance or in monetary value depending on how it is modeled. That translates into:

- faster delivery,
- less battery usage,
- less corridor traffic and disruption,
- more predictable operations.

In my earlier industrial inspired work, the partfeeding problem includes allocating items to tours, capacity constraints and route costs.

For this course and project directive, I would translate it into a hospital logistic scheduling problem

9.1 Hospital Logistics Delivery Replenishment Problem (H-LDRP)

- **Items:** medicine packs, PPE kits, sterile tools, IV bags (each with size/weight and sometimes temperature constraints).
- **Demand points:** wards/rooms with requested quantities and priorities (stat vs routine).
- **Constraints:**
  - robot payload capacity,
  - battery/time budget per tour,

  - optional time windows (medication rounds),
  - safety constraints (avoid certain zones, elevators, infection control routes).

- **minimize a combined cost:**
  - travel time/distance,
  - penalties for late/overdue high-priority deliveries,
  - penalties for too many tours (charging overhead),
  - congestion cost (busy corridor avoidance).

This framing is important because it shows the robot is not just driving. It is implementing an operational solution to a logistics problem: deciding what to deliver, in what order, under constraints, while maintaining safety.

The TSP problem for the shortest part is less complex problem (was demonstrated in lab) than the one proposed here and can be solved relatively easy compared to the this high dimension problem which would require metaheuristics (e.g: PSO, GA) approach for a feasible solution.

Though I did try to solve it in a Matlab environment, I wasn't able to simulate in Gazebo or a live practical demo. As there was no enough physical space to demonstrate it and due to difficulties with Gazebo.

10 Gazebo Simulation attempt

I attempted to model the project in ROS2 Gazebo using the aws-robomaker-hospital-world environment. I did not fully succeed due to a combination of:

1. limited time near the deadline,
2. health/resource constraints
3. integration overhead (Gazebo/ROS2 version mismatches, heavy compute requirements, and sensor plugin tuning).
4. The aws-robomaker-hospital-world model is not directly released for ROS2 Humble or Jazzy unlike the ware house model, I tried to use some bridge for the topics but was unsuccessfull.

with more time, I would:

1. containerize the simulation environment (reproducible setup),
2. run SLAM + navigation against the hospital world,
3. validate the return/avoidance logic under repeatable scenarios,
4. and use simulation to benchmark tuning parameters before hallway tests.

11 Development experience and challenges

Technical Challenges

- Control handover: making sure manual teleop and autonomous return don't fight each other.
- Avoidance edge cases: oscillations, rejoin failures, scan staleness.
- Filtering out the noisy spike in the wheel encoder to obtain a reliable heading.

Process challenges

- Integration always takes longer than expected.
- A feature that works alone can fail when everything runs together

Key learning I learned that robotics is systems engineering:

- small software details (debounce, deadzones, cooldowns) can dominate real-world reliability,
- safety and predictable behavior matter as much as clever algorithms,
- and a working demo is often the result of many small design decisions rather than one big algorithm.

12 Reflections and Proposed Improvements

Hardware/mechanical

- Place the LiDAR closer to the robot axle center to reduce rotational distortion and improve scan-to-motion consistency.
- Add a camera (or depth camera) for better teleoperation feedback and for human-aware navigation later and also so computer vision models can be inherited.
- Improve cable management and mechanical robustness for repeated tests.

Software/architecture

- Make a clearer separation between:
  - low-level velocity control,
  - autonomy states,
  - and mission-level logic (delivery scheduling).
- Add logging and replay tools early (bag files) so tuning becomes data-driven.
- Implement a more explicit map-based planner and local obstacle avoidance layer

Interaction / UX

- Add natural language or task-level telecommands (e.g., "go to Ward B, Room 12") translating into goal lists + TSP scheduling + safe navigation (currently only implemented and tried in MATLAB, but should work when integrated with ROS nodes).

- Improve operator UI with a clean status panel: mode, distance-to-home, scan warnings, estimated battery.

13 Societal and ethical reflections

Societal value A teleoperated/semi-autonomous hospital delivery robot can:

- reduce staff walking workload
- reduce cross-contamination risk by lowering unnecessary human movement,
- improve consistency of routine deliveries,
- allow remote support in off-hours.

Ethical and safety considerations

- **Patient safety**: the robot must never collide with people; speed limiting and conservative avoidance are essential.
- **Privacy**: cameras improve teleop, but they raise privacy concerns in patient areas—data minimization and strict access control would be needed.
- **Accountability**: a human-in-the-loop teleoperation mode is valuable for responsibility and handling corner cases.
- **Reliability and trust**: robots must behave predictably; "sometimes it works" is not acceptable in a medical context.

14 Conclusion

My work contributed strongly to the operational core of the project: reliable teleoperation, return-path behaviors, and safety logic, as well as exploration of planning and estimation methods that can extend the system toward more autonomy. The project reinforced that a meaningful robotics system is not just an action but a robot that solves a real operational problem under constraints—safely, predictably, and with clear interfaces for humans.